

Basic Concept of OOPS

- (1) Object
- (2) Class
- (3) Data Structure & encapsulation
- (4) Inheritance
- (5) Polymorphism
- (6) Dynamic Binding
- (7) Message passing.

(1) Object :- Run time entities

Data + method $\Rightarrow$ combination.

Class student

{

private :

char name[50];

int rollno.;

float fee;

public :

void show();

void input();

};

} Data member
(Data) ..

} member function
(methods)

Object : student	
DATA :	Name DOB Marks
FUNCTION :	Total Avg

(2) Class :- Collection of object

That share some properties of data part & some program part are collectively called a class.

(3) Data abstraction & Incapsulation

Data Abstraction - Abstraction refer to the act of representing essential features without including the background details or explanation.

Incapsulation - The wrapping of Data & function into a single unit is known as a incapsulation. The Data is not accessible to the outside world.

(4) Inheritance :- Process by which one class require the properties of another class. Basically inheritance provides reusability of codes.

(5) Polymorphism :- More then one form function & operation overloading.

add();

add(int a, int b);

(6) Dynamic Binding :- Binding refers to the linking of procedure call to the code to executed response.

(7) Message passing :- Object oriented programming consist a set of object that communicate with each other.

Emp. salary (name)
↙ object → object

* Class

Inside the Class:-

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class Emp
```

```
{
```

```
private:
```

```
char name [30];
```

```
int age;
```

```
public:
```

```
void Input()
```

```
{
```

```
cout << "Input name & Age << endl;
```

```
cin >> name >> age;
```

```
}
```

```
void show()
```

```
{
```

```
cout << "Name & Age : " << endl;
```

```
cout << name << age << endl;
```

```
}
```

```
};
```

```
void main()
```

```
{
```

```
class emp e1;
```

```
e1.Input();
```

```
e1.show();
```

```
getch();
```

```
}
```

Output:

Input Name & age

Ranchan 20

Name & Age

Ranchan 20

class ←

main ←
function

} → object

∴ scope resolution operator

- member access operation.

emp.show
?

Outside the class -

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
class emp
```

```
{
```

```
private:
```

```
char name [30];
```

```
int age;
```

```
public:
```

```
void input();
```

```
void show();
```

```
};
```

```
void emp::input()
```

```
{
```

```
cout << "Input Name & Age << endl;
```

```
cin >> name >> age;
```

```
}
```

```
void emp::show()
```

```
{
```

```
cout << "output is" << endl;
```

```
cout << name << age;
```

```
}
```

```
void main()
```

```
{
```

```
class emp e1;
```

```
e1.input();
```

```
e1.show();
```

```
} getch();
```

output

input name & Age

Kanchan 20

Name & age

Kanchan 20

Q write a Program to construct a class for student where data members should be the name of student, roll number of student & fees of student.

The member function should be input & show. Excess these object in inside the class & outside the class.

Solⁿ (i) Inside the class

```
#include < iostream.h >
```

```
#include < conio.h >
```

```
class student
```

```
{
```

```
private :
```

```
char name [30];
```

```
int rollno.;
```

```
float fees;
```

```
public :
```

```
void input();
```

```
{
```

```
cout << " Input name, roll number & fees of student" << endl;
```

```
cin >> name >> rollno. >> fees;
```

```
}
```

```
void show()
```

```
{
```

```
cout << "Data of student is : " << endl;
```

```
cout << name << rollno. << fees << endl;
```

```
}
```

```
};
```

```

void main ()
{
    class student stu;
    stu.input();
    stu.show();
    getch();
}

```

Now For outside the class

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class student
```

```
{
```

```
private;
```

```
char name [30];
```

```
int rollno;
```

```
float fees;
```

```
public :
```

```
void input ()
```

```
void show ()
```

```
};
```

```
void stu :: input ()
```

```
{
```

```
cout << "Input the name, roll number & fees." << endl;
cin >> name >> rollno >> fees;
```

```
}
```

```
void stu :: show ()
```

```
{
```

```
cout << "Data of student is " << endl;
```

```
cout << name << rollno << fees << endl;
```

```
}
```

```
void main ()
```

```
{
```

```
class student stu;
```

```
stu.input();
```

```
stu.show();
```

```
getch();
```

```
}
```

Array of object :-

Q write a program to display 5 students data using the concept of array of object.

Soln

```
#include <iostream.h>
#include <conio.h>
```

(outside the class)

```
void main()
{
    class student
    {
        char name [50];
        int rollno;
        float fee;
    public:
        void input();
        void show();
    };
}
```

```
void stu::input()
{
    cout << "Input name, roll no. & fee" << endl;
    cin >> name >> rollno. >> fee;
}
```

```
void stu::input show()
{
    cout << "The data of students << endl;
    cout << name << rollno. << fee << endl;
}
```

```

void main ( )
{
    class student stu[5];
    int i;
    for (i=0 ; i<5 ; i++)
    {
        stu[i] = input ();
    }

    cout << "Five students data is = " << endl;
    for (i=0 ; i<5 ; i++)
    {
        stu[i]. show();
    }

    getch();
}

```

For inside the class

```

#include <iostream.h>
#include <conio.h>
class student
{
    char name[50];
    int rollno;
    float fee;

public:
    void input()
    {
        cout << "Input the name, roll no. &
        fees of student" << endl;

        cin >> name >> rollno. >> fee;
    }
}

```

```
cout << endl;  
void show ()
```

```
{  
    cout << name << roll no. << fees << endl;  
}
```

```
void main ()
```

```
{  
    class student stu[5];
```

```
    int i;
```

```
    for (i=0 ; i<5 ; i++)
```

```
    {  
        stu[i].input();
```

```
    }
```

```
    cout << " The data of students : " << endl;
```

```
    for (i=0 ; i<5 ; i++)
```

```
    {  
        stu[i].show();
```

```
    }
```

```
    getch();
```

```
}
```

* Object as argument :

```
#include <iostream.h>
#include <conio.h>
class add addition
{
    private :
        int a, b, c;
    public :
        void input ()
        {
            cout << "Enter the value of a & b" <<
                endl;
            cin >> a >> b;
        }
}
```

```
void display (addition k)
{
    c = k.a + k.b;
    cout << "The sum is" << c << endl;
}
```

```
void main ()
{
    class addition add;
    add.input ();
    add.display (add);
    getch ();
}
```

Enter the value of
a & b

5

7

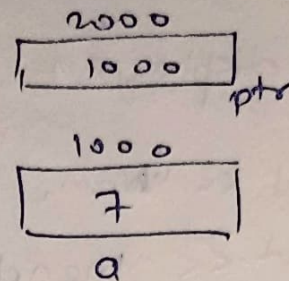
ans is = 12

Note
for call be
address
addition * k
&
(~~add~~)

```

int *ptr;
int a = 7;
ptr = &a;

```



```
printf("%d", a) = 7
```

```
printf("%d", &a) = printf("%d", ptr) = 1000
```

```
printf("%d", *ptr) = 7
```

```
or printf("%d", *(&a)) = 7
```

* class & Pointer

Q Write a Program to display the student data using indirection operator (*) or (→)

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class student
```

```
{
```

```
private:
```

```
char name[50];
```

```
char gender;
```

```
int rollno;
```

```
float height;
```

```
public:
```

```
void Input();
```

```
{
```

```
cout << "enter details" << endl;
```

```
cin >> name >> gender >> rollno >> height;
```

```
}
```

```
void display()
```

```
{  
    cout << "Name is = " << name << endl;  
    cout << "Gender: " << gender << endl;  
    cout << "Roll no. is: " << rollno << endl;  
    cout << "Height is: " << height << endl;  
}
```

3:

```
void main ()
```

```
{  
    class student data;
```

```
    clrscr();
```

```
    class student *ptr;
```

```
    ptr = &data;
```

or for indirection
operator

```
    (*ptr).input();
```

ptr → input();

```
    (*ptr).display();
```

ptr → display();

```
    getch();
```

```
}
```

Output

Enter details -

Kanchan F 21 5.3

Name is Kanchan

Gender is F

Roll no. is 21

Height is 5.3

Q Write a program for class -
class arith

```
{ int a, b;
```

```
public :
```

```
void input();
```

```
display();
```

```
add();
```

```
sub();
```

```
mul();
```

```
};
```

Soln

Ans.

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class arith.
```

```
{
```

```
int a, b;
```

```
public :
```

```
void input();
```

```
void display();
```

```
void add();
```

```
void sub();
```

```
void mul();
```

```
};
```

```
void input arith::input()
```

```
{
```

```
cout << "enter the number a & b" << endl;
```

```
cin >> a >> b;
```

```
void arith::display()
```

```
void arith::add()
```

```
{
```

```
    c = a + b;
```

```
    cout << "Sum is : " << c << endl;
```

```
}
```

```
void arith::sub()
```

```
{
```

```
    c = a - b;
```

```
    cout << "Subtraction is : " << c << endl;
```

```
}
```

```
void arith::mul()
```

```
{
```

```
    c = a * b;
```

```
    cout << "multiplication is : " << c << endl;
```

```
}
```

```
void arith::display()
```

```
{
```

```
    cout << "The output is :-" << endl;
```

```
}
```

```
void main()
```

```
{
```

```
    class arith data;
```

```
    clrscr();
```

```
    class arith ptr;
```

```
    *ptr = &data;
```

```
    (*ptr).input();
```

```
    (*ptr).display();
```

(1) 100,000,000;

(2) 100,000,000;

(3) 100,000,000;

(4) 100,000,000;

?

Access specifier in Object Oriented program

*private :- Data member can be accessed within a class, or we can say private Data member by member function of a class.

Note By default data member should be private.

*public :- Can be accessed from inside the class & outside the class.

*protected :- Some of the protected data member can be accessed from friend function and member function. So